

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART XI

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

Part - I: Overview of Agentic Coding Assistants

Part – II: Claude Ecosystem, Installing & using Claude Code

Part – III: Slash Commands, Context Window & `.claude` Directory

Part – IV: Workflow of Building a Basic Vulnerable Web Application

Part – V: Exploiting Vulnerable Web Application

Part – VI: Workflow of Implementing Features and Fixing Bugs

Part – VII: Collaborative Workflow: Claude Code + GitHub + Jira

Part – VIII: Built-in Skills & Vuln-Web-App Current Status

Part – IX: Creating & using Custom Skills & Skill Marketplace

Part – X: Deep Dive: Conceptual Understanding of MCP

Part – XI: **MCP in Action**

Recap: Model Context Protocol



Some Important MCP Hosts



Claude Code
Anthropic

Claude Desktop
Anthropic

Claude.ai
Anthropic

Claude Cowork
Anthropic

VS Code
Microsoft

Cursor IDE
Anysphere

Antigravity
Google

Windsurf
Codeium

Groq Desktop
Groq

GitHub Copilot
GitHub

MCP Inspector
Model Context Protocol

Goose
Block

https://www.pulsemcp.com/clients

Some Important MCP Servers

Notion
Notion Labs

Jira
Atlassian

Linear
Linear

Google Workspace
Google

GitHub
GitHub

Hugging Face
Hugging Face

Playwright
Microsoft

Context7
Upstash

PostgreSQL
Community

MongoDB
MongoDB

Supabase
Supabase

Redis
Redis

Figma
Figma

Canva
Canva

Stripe
Stripe

PayPal
PayPal

https://www.pulsemcp.com/servers

PART-I

Adding MCP Servers in Claude Desktop

Instructor: Muhammad Arif Butt, PhD

Connecting MCP Servers in Claude Desktop

	REMOTE · IN THE CLOUD	LOCAL · ON YOUR MACHINE
ONE-CLICK	**1 Directory connector** Browse Claude's built-in, reviewed gallery and click Connect — OAuth, nothing to set up. In Claude: Settings → Connectors → Browse EXAMPLE: **Jira** — Atlassian Rovo https://claude.ai/directory/connectors	**3 Desktop Extension (.mcpb)** Install a pre-bundled local server in one click. No setup, no JSON. In Claude: Settings → Extensions → Browse EXAMPLE: **Filesystem** — read/write your folders https://github.com/modelcontextprotocol/mcpb
MANUAL	**2 Custom connector** Paste a remote MCP server's URL yourself — for servers that aren't in the gallery. In Claude: Connectors → + Add custom connector EXAMPLE: **Hugging Face** — huggingface.co/mcp https://registry.modelcontextprotocol.io	**4 Config file** Hand-edit a JSON file to run a local server with full control over command & arguments. In Claude: Settings → Developer → Edit Config EXAMPLE: **Git** — uvx mcp-server-git https://mcpservers.org/

- **Remote connectors** are account-scoped (follow the user across all clients and devices) and work across claude.ai, Cowork, mobile apps, and Claude Desktop.
- **Local servers** are machine-scoped (locked to the specific device where they're installed, and only work in Claude Desktop).

Connecting to Jira via Directory Connector

Connect Claude to your Jira workspace through a one-click connector — then search issues, track sprints, and manage projects without leaving the chat.

How it connects



Connectors are Claude's built-in, reviewed integrations. Atlassian publishes a hosted server in the gallery — connect once via OAuth, so Claude can read and act on Jira *as you*.

What you can do once it's connected

Issues
Search, create, update & transition tickets

Comments
Read & post comments on issues

Sprints & Boards
Track sprint status, boards & backlogs

Projects & JQL
Explore projects & run JQL searches

Setting it up — 3 steps

No tokens, no JSON — just OAuth →

1. **Open Connectors**
   `Settings → Connectors`
2. **Find Jira & Connect**
   `Browse → Atlassian`
3. **Authorize & verify**
   Authenticate → ✓ **Connected**

https://arifpucit.atlassian.net/jira

Connecting to HF via Custom Connector

Connect Claude to Hugging Face through a custom connector — then search the Hub, explore models and datasets, and run community Spaces without leaving the chat.

How it connects



A **custom connector** points Claude at any remote MCP server you trust. Hugging Face hosts one publicly — paste the URL, sign in, and Claude can use the Hub *as you*.

What you can do once it's connected

Models
Search & explore models on the Hub

Datasets
Discover & inspect datasets

Spaces
Run community Spaces as live tools

Papers & Docs
Search papers & documentation

Setting it up — 3 steps

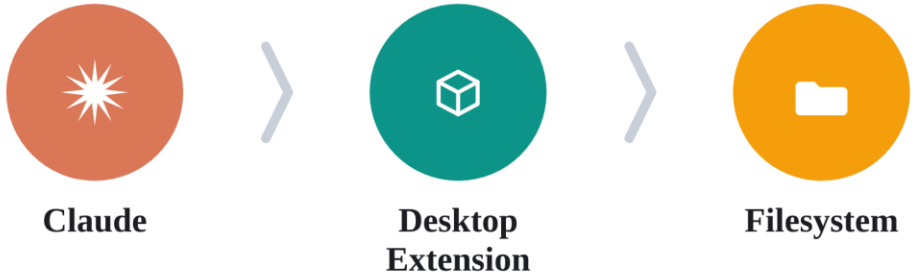
No install, no JSON — just paste the URL →

1. **Open Connectors**
   `Settings → Connectors`
2. **Add custom connector**
   `https://huggingface.co/mcp`
3. **Authorize & verify**
   Sign in → ✓ **Connected**

Connecting to Filesystem via Desktop Extension

Connect Claude to your computer's files through a Desktop Extension — install a one-click .mcpb bundle, then read, write, and organize your folders without leaving the chat.

How it connects



A **Desktop Extension** is a .mcpb bundle — a local MCP server packaged like a browser extension, installed in one click with no terminal or JSON.

What you can do once it's connected

Read files
View the contents of files in your folders

Write & edit
Create and modify files

Move & organize
Rename, move & make folders

Search & list
Find files & browse directory trees

Setting it up — 3 steps

Only install .mcpb bundles you trust →

1. **Get the .mcpb bundle**
   `filesystem.mcpb`
2. **Install it in Claude**
   `Drag onto Settings → Extensions or Advanced → Install Extension…`
3. **Pick folders & verify**
   Choose your folders → ✓ **Installed**

https://github.com/modelcontextprotocol/servers

Connecting to Git via Config File

Connect Claude to a Git repository through the config file — add one JSON entry by hand, then explore history, diffs, and branches without leaving the chat.

How it connects



The **config file** route adds any MCP server by hand — you edit Claude's JSON directly. No gallery, full control over the command.

What you can do once it's connected

Status & log
See working-tree status & history

Diffs
View staged & unstaged changes

Commit & stage
Stage files and create commits

Branches
Create, switch & compare branches

Setting it up — 3 steps

Needs uv + Git installed →

1. **Open the config file**
   `Settings → Developer → Edit Config`
2. **Paste the Git block**
   Change the repo path to your folder
3. **Save & restart Claude**
   Git tools appear → ✓ **Connected**

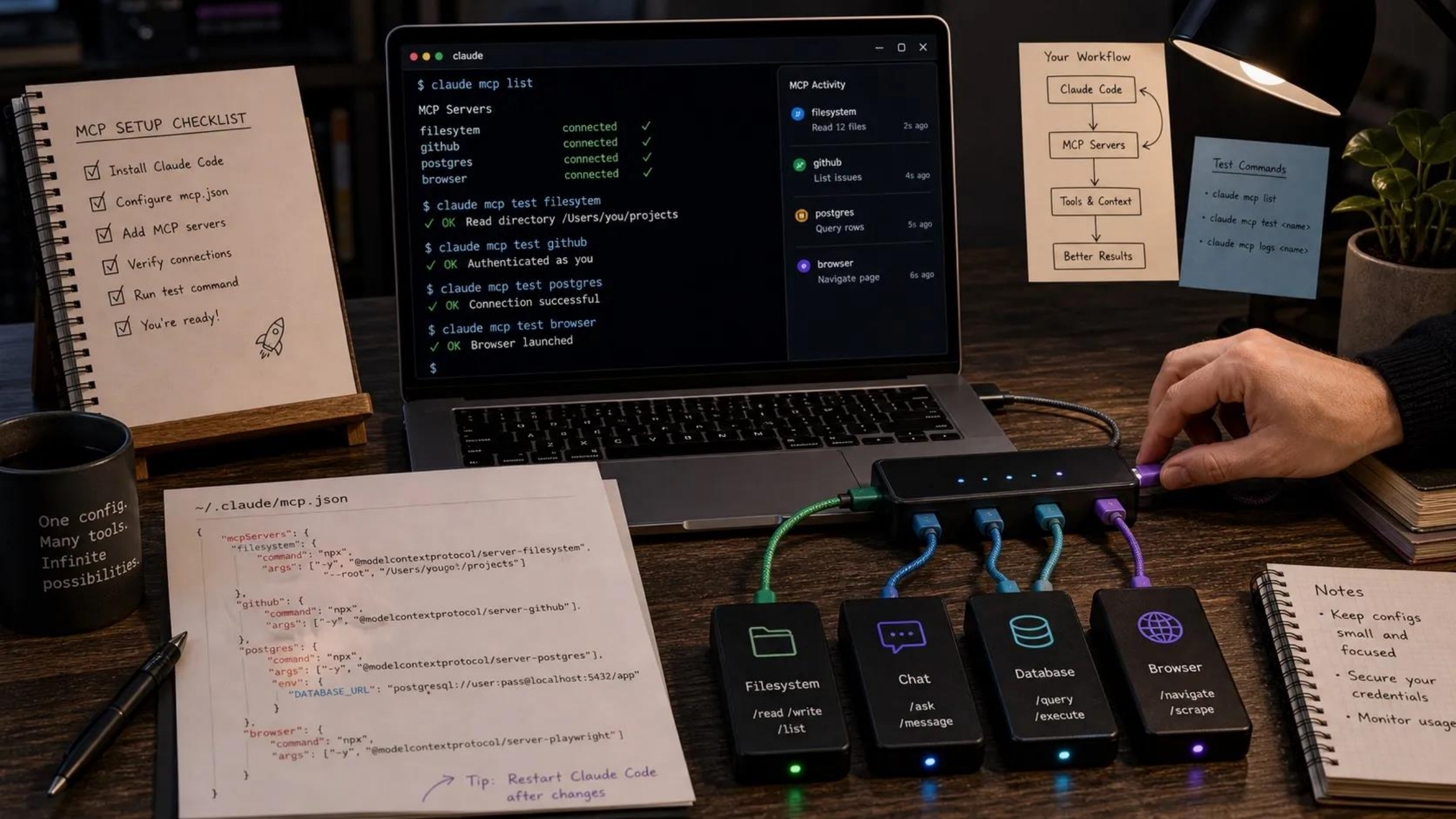
claude_desktop_config.json

```
{
  "mcpServers": {
    "git": {
      "command": "uvx",
      "args": ["mcp-server-git", "--repository", "/path/to/your/repo"]
    }
  }
}
```

PART-II

Adding MCP Servers in Claude Code

Instructor: Muhammad Arif Butt, PhD


MCP SETUP CHECKLIST
Install Claude Code
Configure mcp.json
Add MCP servers
Verify connections
Run test command
You're ready!
claude
$ claude mcp list
MCP Servers
filesytem connected
github connected
postgres connected
browser connected
$ claude mcp test filesytem
OK Read directory /Users/you/projects
$ claude mcp test github
OK Authenticated as you
$ claude mcp test postgres
OK Connection successful
$ claude mcp test browser
OK Browser launched
$
MCP Activity
filesystem
Read 12 files
2s ago
github
List issues
4s ago
postgres
Query rows
5s ago
browser
Navigate page
6s ago
Your Workflow
Claude Code
MCP Servers
Tools & Context
Better Results
Test Commands
• claude mcp list
• claude mcp test <name>
• claude mcp logs <name>
One config.
Many tools.
Infinite
possibilities.
~/.claude/mcp.json
{
"mcpServers": {
"filesystem": {
"command": "npx",
"args": ["-y", "@modelcontextprotocol/server-filesystem",
"--root", "/Users/you/projects"]
},
"github": {
"command": "npx",
"args": ["-y", "@modelcontextprotocol/server-github"]
},
"postgres": {
"command": "npx",
"args": ["-y", "@modelcontextprotocol/server-postgres"],
"env": {
"DATABASE_URL": "postgresql://user:pass@localhost:5432/app"
}
},
"browser": {
"command": "npx",
"args": ["-y", "@modelcontextprotocol/server-playwright"]
}
}
Tip: Restart Claude Code after changes
Filesystem
/read /write
/list
Chat
/ask
/message
Database
/query
/execute
Browser
/navigate
/scrape
Notes
• Keep configs small and focused
• Secure your credentials
• Monitor usage

Connecting MCP Servers In Claude Code

1 STDIO · LOCAL

Local server (stdio)

Runs as a process on your machine

```
$ claude mcp add
    --transport stdio
    <name> --
    <command> [args...]
# stdio is the default
```

2 HTTP · REMOTE

Remote server (HTTP)

Connects to a server on the internet

```
$ claude mcp add
    --transport http
    <name> <url>

# e.g. https://mcp.example.com/mcp
```

3 STDIO + HTTP

Add from JSON config

Paste a ready-made config from vendor docs

```
$ claude mcp add-json
    <name> '<json>'

# "type": "stdio"
# "type": "http"
```

4 STDIO + HTTP

Edit config files directly

Hand-edit the files where servers live

```
~/.claude.json
  # local & user scope

.mcp.json
  # project scope
```

5 **Import from Claude Desktop** STDIO + HTTP

Reuse the servers already configured in claude_desktop_config.json

```
$ claude mcp add-from-claude-desktop
```

`--scope` **local** = default, this project only • **project** = shared via .mcp.json • **user** = all your projects

Adding GitHub MCP Server in Claude Code

Connect Claude Code to your GitHub account through MCP — then work with repos, issues, and pull requests without leaving your terminal.

1 **Create GitHub PAT:** https://github.com/settings/profile

claude — zsh github MCP

```
$ claude mcp add --transport http github \
    https://api.githubcopilot.com/mcp/ \
   --header "Authorization: Bearer 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" \
   --scope user
✓ Added HTTP MCP server "github"
# restart Claude Code, then:
$ claude mcp list
  github   ✓ Connected
```

2 **Add the server** → **3** **Restart Claude Code** → **4** `/mcp` → **✓ Connected**

Not showing Connected yet?

In Claude Code, run `/mcp` → select **`github`** → **Authenticate**. Claude opens your browser to sign in to GitHub — approve it, and the server connects.

Add it, restart Claude Code, and confirm the status reads **✓ Connected**.

Adding Google MCP Server in Claude Code

Step 1 · Google Cloud Setup

Create your project, enable the APIs, and open the Google Auth Platform.

1 Create a Project

Open the Google Cloud Console and sign in. From the project selector at the top-left, click **New Project**, give it a name, and click **Create**.

console.cloud.google.com

2 Enable the Gmail / Calendar / Drive API

With the **correct project selected**, open the API Library below, search for Gmail (also Calendar / Drive), and click **ENABLE** (allow 1–2 minutes to take effect).

console.cloud.google.com/apis/library

3 Open the Google Auth Platform

Visit the page below — it will say the *Google Auth Platform is not configured yet*. Make sure your project is selected at the top, then click **Get Started**.

console.cloud.google.com/auth/overview

Adding Google MCP Server in Claude Code

Step 1 · Google Cloud Setup (contd.)

Configure the app, create the OAuth client, publish, and copy your credentials.

4 Complete the Project Configuration

Fill in the configuration form: enter an app name, and when asked for a support email, choose the **email you are currently signed in with**. Accept the terms and click **Create**.

5 Create the OAuth Client ID

Under the **Metrics** section you will see "You haven't configured any OAuth clients for this project yet." Click **Create OAuth client**, set the application type to **Desktop app**, give it a name of your choice, and click **Create**.

6 Publish the App

In the left panel, open the **Audience** tab. Under **Publishing status**, click **Publish app**.

⚠ *Don't leave the app in Testing — login will fail. Publishing fixes it.*

7 Copy the Client ID & Secret

Open the clients page below and select your project. Click your client the **Client ID** and **Client secret** appear on the right side. Copy both.

console.cloud.google.com/auth/clients

Adding Google MCP Server in Claude Code

Step 2 · Add the Server

Register the Google Workspace MCP server in Claude Code via the config file, using your OAuth keys from Step 1.

1 Install uv

The server runs via **uvx**, which comes with **uv**. Check it, and install it if missing:

```
uv --version
```

```
pip install uv
```

2 Open the Config File in Your Home Directory

Open the **.claude.json** file in your home directory. We register the server here, under the main **mcpServers** key, so that it is available from **any directory** on your machine:

```
C:\Users\<YourName>\.claude.json
```

3 Add the Server Block, Save & Restart

Paste this under **mcpServers**, replacing **YOUR_ID** / **YOUR_SECRET** with your keys from Step 1:

```
"google-workspace": {
  "command": " "/Users/arif/.local/bin/uvx",
  "args": ["workspace-mcp", "--single-user", "--tools", "gmail", "calendar", "drive"],
  "env": {
    "GOOGLE_OAUTH_CLIENT_ID": "YOUR_ID",
    "GOOGLE_OAUTH_CLIENT_SECRET": "YOUR_SECRET",
    "OAUTHLIB_INSECURE_TRANSPORT": "1"}}
```

PART-III

Creating/Adding your own MCP Server

Instructor: Muhammad Arif Butt, PhD

Creating MCP server

Our Demo Server at a Glance

Demo-server (main.py) exposes seven tools and two resources — this is what we will test.

Tools (7)

`add_numbers` — add two numbers and return the sum

`roll_dice` — roll n six-sided dice

`get_datetime` — current date and time

`fahrenheit_to_celsius` — convert °F to °C

`reverse_string` — reverse any text

`count_words` — count words in a text

`get_weather` — live weather via WeatherAPI

https://github.com/arifpucit/mcp-server

Resources (2)

`local://greeting`
a welcome message from the server

`local://sample-data`
sample students JSON data

Setup Note

`get_weather` reads `WEATHER_API_KEY` from a .env file (python-dotenv).

What is MCP Inspector?

A local debugging tool to inspect and exercise your MCP server before wiring it into any client.

Nine things the Inspector lets you do once your server is running.

1

Connect to Server

Attach to your server over STDIO or SSE transport.

2

Initialize Session

Start a session and verify the connection is healthy.

3

List Tools

Discover and view every tool the server exposes.

4

List Resources

Browse all resources the server makes available.

5

List Prompts

View available prompts and their descriptions.

6

Call Tools

Run tools with custom inputs and see live responses.

7

Inspect Responses

Examine structured replies, logs, and any errors.

8

View Server Info

Check capabilities, protocol version, and metadata.

9

Debug & Test

Exercise every feature locally before you ship.

Test Demo-server with the Inspector

Launch the server in dev mode with the Inspector attached, then exercise it from the browser.

```
# launch the Inspector against your server
$ uv run fastmcp dev inspector --server-spec main.py
# → opens in your browser at localhost:6274
```

Connect

The Inspector opens already connected — a live session over STDIO transport.

Verify

The Tools tab lists all 7 tools; the Resources tab shows both resources.

Call

Run `get_weather("Lahore")` with your own input and inspect the live JSON reply.

Deploy the Server & Add it to Claude Code

Start the server in run mode, register it with Claude Code, then verify the connection.

```
# 1. run the server (plain STDIO — it waits silently for a client)
$ uv run fastmcp run main.py

# 2. register Demo-server with Claude Code
$ claude mcp add Demo-server -- uv run --directory /Users/arif/mcp-server fastmcp run main.py

# 3. verify the connection
$ claude mcp list          Demo-server ✓ connected
```

dev attaches the Inspector for debugging; **run** starts a plain STDIO server that real clients connect to.

Demo-server is the name; everything after **--** is the launch command Claude Code runs to start your server.

Use it in Claude Code

Ask in plain English — Claude Code picks the right tools from Demo-server automatically.

```
> Roll 3 dice and tell me the current weather in Lahore

● roll_dice(n=3)  →  [4, 2, 6]
● get_weather(city="Lahore")  →  31°C, Partly cloudy

● You rolled 4, 2 and 6 (total 12). The weather in Lahore
is currently 31°C and partly cloudy.
```

Check anytime

Type **/mcp** inside Claude Code to view connected servers and their tools.

The full loop

Build main.py → **Inspect** locally → **Run** the server → **Connect** it to Claude Code.

PART-IV

Creating a Skill with MCP Server

Instructor: Muhammad Arif Butt, PhD

The Six Pillars of the Claude Code Ecosystem

Knowledge — What Claude knows

CLAUDE.md

ALWAYS LOADED

Persistent project context loaded at the start of every session. Files merge hierarchically — user, project, subdirectory — with more specific files winning.

Skills

USER OR CLAUDE INVOKED

Reusable instructions plus bundled scripts and assets, loaded only when needed via progressive disclosure. Run manually with /skill-name or let Claude trigger one from its description. Absorbed custom slash commands.

Execution — How work gets done

MCP servers

MODEL INVOKED

Brokered access to external systems — APIs, databases, and developer tools — exposed to Claude as callable tools over a standard protocol.

Subagents

DELEGATED

Specialized workers that run in their own isolated context window and return only a summary — heavy research and parallel tasks never pollute the main thread.

Operations — Enforcing and sharing

Hooks

EVENT TRIGGERED

Deterministic code that fires on lifecycle events — formatting, logging, validation, notifications. Enforce rules with code, not prompts.

Plugins

DISTRIBUTION UNIT

Bundles skills, subagents, hooks, MCP configs, and LSP servers* into one installable, versioned package — the canonical way to share a setup across teams and marketplaces.

* LSP servers give Claude IDE-grade code intelligence — go-to-definition, find references, live diagnostics — via the Language Server Protocol. Configured through plugins; the language server binary is installed separately.

The Repository: arif-skills

github.com/arifpucit/arif-skills

```
arif-skills/
├── CLAUDE.md
├── .mcp.json
├── .claude/  hidden — ls -a
│   └── skills/
│       └── repo-report/  ← slide 2
│           ├── SKILL.md
│           ├── references/
│           ├── scripts/
│           └── assets/
├── examples/
├── check-setup.sh
├── README.md
└── LICENSE, .gitignore
```

CLAUDE.md — project memory

Loaded at the start of every session: what this repo is, where skills live, conventions to preserve, and a smoke-test command. Facts and conventions live here — procedures belong in skills.

.mcp.json — shared MCP config

Defines the remote GitHub MCP server at project scope, so every clone gets it automatically. Auth comes from the GITHUB_PAT environment variable — the committed file contains no secrets. Students approve it once at first launch.

.claude/skills/ — where skills live

A hidden folder Claude Code reads (ls -a to see it). One subdirectory per skill; repo-report is the first. Add a new skill, push — every student who pulls gets it. Its anatomy is the next slide.

Student onramp

README.md walks prerequisites, setup and a troubleshooting table. check-setup.sh verifies CLI, token and files — five passes means ready. examples/ holds sample invocations plus deliberate failure exercises for class.

```
git clone https://github.com/arifpucit/arif-skills.git
```

One-time setup per machine — then every skill in the repo just works.

git clone the repo → export GITHUB_PAT → connect gmail MCP → check-setup.sh 5 passes → claude approve 2 prompts

Anatomy of repo-report Skill

`.claude/skills/repo-report/`	FOLDER	The skill's root inside the repo. The directory name becomes the command — /repo-report — and the description lets Claude invoke it automatically.
├── **SKILL.md**	REQUIRED	The entrypoint. YAML frontmatter declares description, arguments (repo_url, recipient) and allowed-tools pre-approving mcp__github and mcp__gmail; the body holds the step-by-step workflow.
├── README.md	DOC	For humans only — install paths, invocation examples, customization notes. Claude never loads it.
├── **references/**	FOLDER	Detailed guidance read on demand — progressive disclosure keeps it out of context until the skill runs.
│ ├── github-data-mapping.md	DOC	Maps each metric (forks, branches, tags, PRs, releases, collaborators & tasks) to an MCP tool intent plus a fallback; honesty rules: capped counts shown as N+, derived values footnoted.
│ └── email-template.md	DOC	Subject format, section order, plain-text table style under 78 chars, and tone rules for the emailed report.
├── **scripts/**	FOLDER	Executable helpers Claude can run with the Bash tool during the workflow.
│ └── parse_repo.sh	SCRIPT	Normalizes any GitHub reference — https, ssh, .git, deep paths, or plain owner/repo — into OWNER REPO; rejects invalid input.
└── **assets/**	FOLDER	Static resources used when producing output.
└── report-style.html	ASSET	Email-safe HTML wrapper (inline styles) for the report tables; used when the Gmail MCP server supports HTML bodies.

/repo-report <repo-url> <email> [metrics…]

Anything after the email is the requested metric list; omit it for the full default report.



OWASP MCP Top 10

1 Token Mismanagement & Secret Exposure

Hard-coded or long-lived keys and tokens can leak through logs or context, giving attackers access to connected systems.

MCP01:2025

2 Privilege Escalation via Scope Creep

Permissions that start too broad or quietly grow over time let agents — or attackers — do far more than intended.

MCP02:2025

3 Tool Poisoning

Malicious instructions hidden in a tool's description, schema, or output can trick the model into harmful actions.

MCP03:2025

4 Software Supply Chain Attacks & Dependency Tampering

Compromised packages, connectors, or dependencies can introduce backdoors or silently change how agents behave.

MCP04:2025

5 Command Injection & Execution

Untrusted input fed into shell commands, scripts, or code can let attackers run arbitrary commands on the host.

MCP05:2025

6 Prompt Injection via Contextual Payloads

Malicious text smuggled into prompts or fetched data can override instructions and make the model misbehave.

MCP06:2025

7 Insufficient Authentication & Authorization

Weak or missing identity and access checks let the wrong users or services reach tools, data, and actions.

MCP07:2025

8 Lack of Audit and Telemetry

Without detailed, tamper-proof logs, attacks and abnormal behavior are hard to detect, investigate, or stop.

MCP08:2025

9 Shadow MCP Servers

Unapproved, unmonitored MCP servers running outside official oversight quietly expand the attack surface.

MCP09:2025

10 Context Injection & Over-Sharing

Shared or persistent context windows can leak sensitive data across users, tasks, or sessions.

MCP10:2025

What you grant a connected MCP server

- **Files, APIs, databases & cloud services** it is wired to (GitHub, Slack, Drive…).
- **Local (stdio) servers inherit your OS user** — your whole file system & OS.
- **Remote (HTTP) servers use stored credentials** — PATs or OAuth tokens.
- **Every server you connect** widens the trust boundary and attack surface.

Developer steps to secure its use

- **Apply least privilege; prefer read-only** access, especially for demos.
- **Review what each server can actually do** before you connect it.
- **OAuth 2.1 + PKCE, short-lived scoped tokens** — no static keys.
- **Keep secrets out of** plaintext config, logs & version control.
- **Treat tool output as untrusted;** sandbox servers & generated code.
- **Human-approve risky actions;** inventory, log & monitor everything.

Sources: MCP spec (modelcontextprotocol.io), OWASP Gen AI Security Project & CoSAI, 2026

Q/A



Thank you!